

Vorwort

Die Quanteninformatik ist eines der spannendsten und zugleich herausforderndsten Gebiete der modernen Wissenschaft. Während klassische Computer auf Bits basieren, die nur die Zustände 0 oder 1 annehmen können, eröffnet die Quantenmechanik mit Qubits, Superposition und Verschränkung völlig neue Möglichkeiten der Informationsverarbeitung.

Dieses Skript richtet sich an Leserinnen und Leser, die einen praxisnahen, verständlichen und mathematisch sauberen Einstieg in die Grundlagen des Quantumcomputings suchen – ohne dabei in die tiefen Bereiche der Quantenphysik abzutauschen. Der Fokus liegt auf den informatikrelevanten Konzepten, die notwendig sind, um Quantenalgorithmen zu verstehen und eigene Simulationen zu entwickeln.

Alle Kapitel sind so aufgebaut, dass sie sowohl Einsteigern als auch technisch versierten Lesern einen klaren Zugang ermöglichen. Die Inhalte sind bewusst kompakt gehalten, aber fachlich korrekt und vollständig genug, um ein solides Fundament zu schaffen.

Dieses Skript entstand aus dem Wunsch heraus, die theoretischen Grundlagen mit einer praktischen Implementierung zu verbinden. Die begleitenden C#-Beispiele zeigen, wie sich Quantenregister, Gatter, GHZ- und Bell-Zustände sowie ein Quanten-Zufallszahlengenerator effizient simulieren lassen.

Ich hoffe, dass dieses Dokument nicht nur Wissen vermittelt, sondern auch Begeisterung für die faszinierende Welt der Quanteninformatik weckt.

Michele Natale, Schweiz 2026

Inhaltsverzeichnis

Vorwort

Kapitel 1: Qubits

- 1.1 Was ist ein Qubit? (Intuitive Erklärung)
- 1.2 Mathematische Darstellung eines Qubits
- 1.3 ASCII Diagramm (für GitHub)
- 1.4 Messung eines Qubits
- 1.5 Beispiel
- 1.6 Bezug zum Framework (C#)

Kapitel 2: Amplituden

- 2.1 Was sind Amplituden? (Intuitive Erklärung)
- 2.2 Warum sind Amplituden komplex?
- 2.3 Mathematische Darstellung
- 2.4 ASCII Diagramm: Amplituden als Pfeile
- 2.5 Normierung
- 2.6 Warum ist Normierung wichtig?
- 2.7 Beispiel: Amplituden eines Bell-States
- 2.8 Bezug zum Framework

Kapitel 3: Superposition

- 3.1 Was bedeutet Superposition? (Intuitive Erklärung)
- 3.2 Warum ist Superposition so wichtig?
- 3.3 Mathematische Darstellung
- 3.4 ASCII Diagramm: Superposition auf der Bloch Kugel (vereinfacht)
- 3.5 Wie erzeugt man Superposition?
- 3.6 Interferenz – das Geheimnis der Quantenmechanik

3.7 Beispiel: Superposition im Framework

3.8 Superposition ist kein „gleichzeitig 0 und 1“

Kapitel 4: Tensorprodukte

4.1 Intuitive Erklärung

4.2 Warum Tensorprodukt?

4.2.1 Was das Tensorprodukt physikalisch bedeutet

- 1) Das Tensorprodukt kombiniert Qubits zu einem einzigen Gesamtzustand
- 2) Das Tensorprodukt definiert, was Verschränkung überhaupt ist
- 3) Das Tensorprodukt erzeugt die exponentielle Dimension (2^n)

4.3 Mathematische Definition

4.4 ASCII Diagramm: Tensorprodukt

4.5 Beispiel: Zwei Hadamard-Qubits

4.6 Tensorprodukt im Framework

4.7 Warum Tensorprodukte Verschränkung ermöglichen

4.8 Beispiel: GHZ Zustand

4.9 Kriterium für Produktzustand bzw. Verschränkung bei zwei Qubits

Kriterium

Beispiel: Produktzustand

Kann man aus einem Produktzustand eine Verschränkung machen?

Kann man aus einer Verschränkung wieder ein Tensorprodukt machen?

Kurz gesagt

Kapitel 5: Quantenregister

5.1 Was ist ein Quantenregister? (Intuitive Erklärung)

5.2 Warum hat ein n Qubit Register 2^n Amplituden?

5.3 Mathematische Darstellung eines Registers

5.4 ASCII Diagramm: Register und Basiszustände

5.5 Wie speichert man ein Register in der Simulation?

5.6 Normierung des Registers

5.7 Messung eines Registers

5.8 Warum Register so wichtig sind

5.9 Beispiel: Ein 3-Qubit-Register im Framework

Kapitel 6: Quantengatter

6.1 Was ist ein Quanten-Gate? (Intuitive Erklärung)

6.2 Warum müssen Quanten-Gatter unitär sein?

6.3 1-Qubit-Gatter

6.4 Wichtige 1-Qubit-Gatter

Hadamard-Gate (H)

Pauli X (NOT Gate)

Pauli Z (Phasenflip)

6.5 2-Qubit-Gatter

CNOT (Controlled-NOT)

6.6 3-Qubit-Gatter

Toffoli (CCNOT)

CCZ (Controlled-Controlled-Z)

6.7 ASCII-Diagramme für GitHub

Hadamard

CNOT

Toffoli

6.8 Wie wirken Gatter auf Register?

6.9 Gatter im Framework

Hadamard

Pauli-X

Pauli-Z

CNOT

Kapitel 7: Messungen (Measure)

7.1 Was bedeutet Messung? (Intuitive Erklärung)

7.2 Warum kollabiert der Zustand?

7.3 Messung eines n-Qubit-Registers

7.4 ASCII Diagramm: Messung

7.5 Warum ist Messung irreversibel?

7.6 Beispiel: Messung eines Bell-Zustands

7.7 Beispiel: Messung eines GHZ-Zustands

7.8 Messung im Framework

- Ganzes Register messen

- Bits extrahieren

7.9 Warum Messung so wichtig ist

Kapitel 8: GHZ- und Bell-Zustände

8.1 Was ist Verschränkung? (Intuitive Erklärung)

8.2 Warum ist Verschränkung so besonders?

8.3 Die vier Bell Zustände

8.4 ASCII-Diagramm: Erzeugung eines Bell-Zustands

8.5 Messung eines Bell-Zustands

8.6 Bell-Zustände im Framework

8.7 GHZ-Zustände (Mehr-Qubit-Verschränkung)

8.8 ASCII-Diagramm: GHZ-Erzeugung

8.9 GHZ-Zustände im Framework

8.10 Messung eines GHZ-Zustands

8.11 Warum GHZ und Bell so wichtig sind

Kapitel 9: Quanten-Zufallszahlen (QRNG)

9.1 Warum Quantenmechanik echten Zufall liefert

9.2 Wie erzeugt man Quanten-Zufall?

- 9.3 QRNG mit mehreren Qubits
- 9.4 QPRNG im Framework (einfachste Version)
- 9.5 QRNG mit GHZ Zuständen
- 9.6 QRNG mit Bell-Zuständen
- 9.7 Warum QRNG so wichtig ist
- 9.8 Beispiel: 5-Qubit-QPRNG
- 9.9 QPRNG als Baustein für das GitHub Projekt

Kapitel 10: Simulationstechniken

- 10.1 Warum ist Quanten-Simulation schwierig?
- 10.2 Der Zustandsvektor
- 10.3 Anwendung von Gattern (naiv)
- 10.4 Sparse-Simulation (effizient)
 - 1-Qubit-Gatter
 - 2-Qubit-Gatter (z. B. CNOT)
- 10.5 Permutationsgatter
- 10.6 Phasengatter
- 10.7 Parallelisierung
- 10.8 Speicheroptimierung
- 10.9 Grenzen der Simulation
- 10.10 Simulation im Framework
 - State-Vector-Simulation
 - Sparse-Gates
 - Permutation & Phase
 - Effiziente Messung
 - GHZ, Bell, QRNG

Schlusswort

Autoren- und Quellenhinweis

1.1 Was ist ein Qubit? (Intuitive Erklärung)

Ein Qubit ist das quantenmechanische Gegenstück zum klassischen Bit, das nur zwei Zustände annehmen kann.

0 oder 1

Ein Qubit kann darüber hinaus zusätzlich in einer Überlagerung (Superposition) dieser beiden Zustände existieren.

Das bedeutet:

Ein Qubit kann gleichzeitig „ein bisschen 0“ und „ein bisschen 1“ sein.

Das ist keine Magie, das die Quantenmechanik ausmacht.

1.2 Mathematische Darstellung eines Qubits

Ein Qubit wird als Vektor in einem zweidimensionalen komplexen Raum dargestellt:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Dabei sind:

- α und β komplexe Zahlen
- $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
- $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

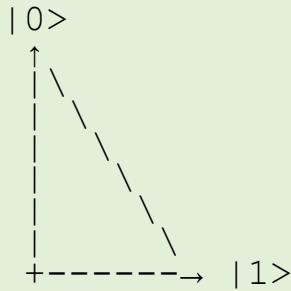
Und ganz wichtig:

$$|\alpha|^2 + |\beta|^2 = 1$$

Das ist die **Normierungsbedingung** – sie stellt sicher, dass die Gesamtwahrscheinlichkeit 1 ist.

1.3 ASCII-Diagramm (für GitHub)

Qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$



Ein Punkt auf dieser Linie repräsentiert den Zustand des Qubits.

1.4 Messung eines Qubits

Wird ein Qubit gemessen, passiert Folgendes:

- Mit Wahrscheinlichkeit $|\alpha|^2$ bekommst man **0**
- Mit Wahrscheinlichkeit $|\beta|^2$ bekommst man **1**

Nach der Messung **kollabiert** („springt“) das Qubit in den gemessenen Zustand. Der mathematische Kollaps ist ein **diskreter Vorgang**.

$$a|0\rangle + b|1\rangle \rightarrow \text{Messung} \rightarrow |0\rangle \text{ oder } |1\rangle$$

Das Qubit springt also tatsächlich in einen der beiden Basiszustände $|0\rangle$ oder $|1\rangle$, wobei nachträglich **alle ursprüngliche Informationen** zur Superposition verloren gehen.

1.5 Beispiel

Ein Qubit im Zustand (Superposition):

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

liefert bei Messung:

- 50 % $\rightarrow$ 0
- 50 % $\rightarrow$ 1

Das ist der Zustand, den wir mit dem **Hadamard-Gate** erzeugen.

1.6 Bezug zum Framework (C#)

Im Framework entspricht ein Qubit:

- einem **2-Elemente-Array** von komplexen Zahlen
- oder einem **Index** in einem grösseren Register

Beispiel:

```
// |ψ> = α|0> + β|1>  
Cx[] qubit = new Cx[]  
{  
    new Cx(alphaRe, alphaIm),  
    new Cx(betaRe, betaIm)  
};
```

In einem Register mit mehreren Qubits ist ein einzelnes Qubit nicht isoliert – es ist Teil eines **Tensorprodukts**.

Das wird im nächsten Kapitel behandelt.

Kapitel 2: Amplituden

2.1 Was sind Amplituden? (Intuitive Erklärung)

Wenn ein Qubit in einem Zustand wie

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

steht, dann sind α und β die sogenannten **Amplituden**.

Sie sind:

- keine Wahrscheinlichkeiten
- keine „Gewichte“
- keine „Anteile“

sondern etwas viel Interessanteres:

Amplituden sind die mathematischen Grössen, aus denen Wahrscheinlichkeiten entstehen.

Die Wahrscheinlichkeit, bei einer Messung **0** zu erhalten, ist:

$$|\alpha|^2$$

Die Wahrscheinlichkeit, **1** zu erhalten, ist:

$$|\beta|^2$$

2.2 Warum sind Amplituden komplex?

Das ist eine der Fragen, die viele Einsteiger verwirrt.

Die kurze Antwort:

Weil Quantenmechanik Interferenz braucht – und Interferenz funktioniert nur mit komplexen Zahlen.

Komplexe Zahlen erlauben:

- Phasen
- Rotationen
- Interferenz (konstruktiv und destruktiv)
- elegante mathematische Darstellung

Ohne komplexe Zahlen gäbe es:

- keine Hadamard-Interferenz
- keine Quantenalgorithmen
- keine Beschleunigung gegenüber klassischen Computern

2.3 Mathematische Darstellung

Ein Qubit:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

mit:

- $\alpha, \beta \in \mathbb{C}$
- $|\alpha|^2 + |\beta|^2 = 1$

Ein Register mit **n Qubits** hat **2^n Amplituden**:

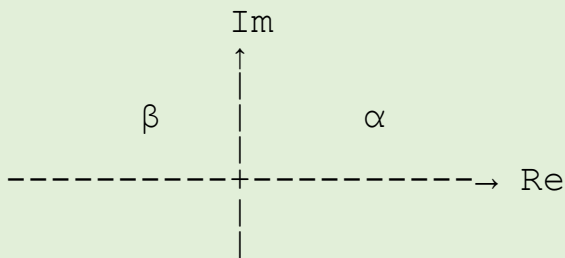
$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle$$

Beispiel für 3 Qubits, wobei z.B. $|001\rangle = |0\rangle|0\rangle|1\rangle$ ist. (Kurzform Tensorprodukt):

$$|\Psi\rangle = a_0|000\rangle + a_1|001\rangle + \dots + a_7|111\rangle$$

2.4 ASCII-Diagramm: Amplituden als Pfeile

Komplexe Ebene:



Jede Amplitude ist ein **Pfeil** in der komplexen Ebene:

- Länge → beeinflusst Wahrscheinlichkeit
- Winkel → beeinflusst Interferenz

2.5 Normierung

Die Summe aller Wahrscheinlichkeiten muss 1 sein:

$$\sum_i |\alpha_i|^2 = 1$$

Das ist die **Normierungsbedingung**.

Im Framework:

```
double sum = 0;
for (int i = 0; i < amps.Length; i++)
    sum += amps[i].NormSquared;

double inv = 1.0 / Math.Sqrt(sum);

for (int i = 0; i < amps.Length; i++)
```

```
amps[i] = amps[i].Scale(inv);
```

2.6 Warum ist Normierung wichtig?

Weil ein Quantencomputer ein **Wahrscheinlichkeitssystem** ist.

Wenn die Summe der Wahrscheinlichkeiten $\neq 1$ wäre:

- Messungen wären unphysikalisch
- Algorithmen würden falsche Ergebnisse liefern
- Interferenz würde nicht funktionieren

Normierung ist also **Pflicht**.

2.7 Beispiel: Amplituden eines Bell-States

Bell-State:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Amplituden:

$$\begin{aligned}a_{00} &= 1/\sqrt{2} \\ a_{01} &= 0 \\ a_{10} &= 0 \\ a_{11} &= 1/\sqrt{2}\end{aligned}$$

Wahrscheinlichkeiten:

$$\begin{aligned}P(00) &= 1/2 \\ P(11) &= 1/2\end{aligned}$$

2.8 Bezug zum Framework

Im universellen Register:

- *Amplitudes[i]* ist die Amplitude des Basiszustands $|i\rangle$
- i ist ein Integer von 0 bis $2^n - 1$
- die Bits von i entsprechen den Qubits

Beispiel:

$$i = 5 = 101_2 \rightarrow |q_2 q_1 q_0\rangle = |1 0 1\rangle$$

Das ist exakt die Struktur, die man braucht, um:

- Gates generisch anzuwenden
- GHZ-Zustände zu erzeugen
- QRNG zu bauen
- Simulationen zu machen

Kapitel 3: Superposition

3.1 Was bedeutet Superposition? (Intuitive Erklärung)

Ein klassisches Bit kann nur **0** oder **1** sein.

Ein Qubit kann zusätzlich in einer **Überlagerung** dieser beiden Zustände existieren:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Das bedeutet NICHT:

- dass das Qubit „halb 0, halb 1“ ist
- oder dass es „zwischen 0 und 1“ steht
- oder dass es „unscharf“ ist

Sondern:

Ein Qubit befindet sich in einem Zustand, der beide Möglichkeiten gleichzeitig enthält – aber **erst die Messung** entscheidet, welche Realität sichtbar wird.

Superposition ist also **kein grauer Mischzustand**, sondern ein **präziser mathematischer Zustand**.

3.2 Warum ist Superposition so wichtig?

Weil sie die Grundlage für:

- Interferenz

- Quantenparallelität
- Quantenalgorithmen
- Bell-Zustände
- GHZ-Zustände
- Quantenrandomness

ist.

Ohne Superposition gäbe es **keinen einzigen Vorteil** gegenüber klassischen Computern.

3.3 Mathematische Darstellung

Ein Qubit in Superposition:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

mit:

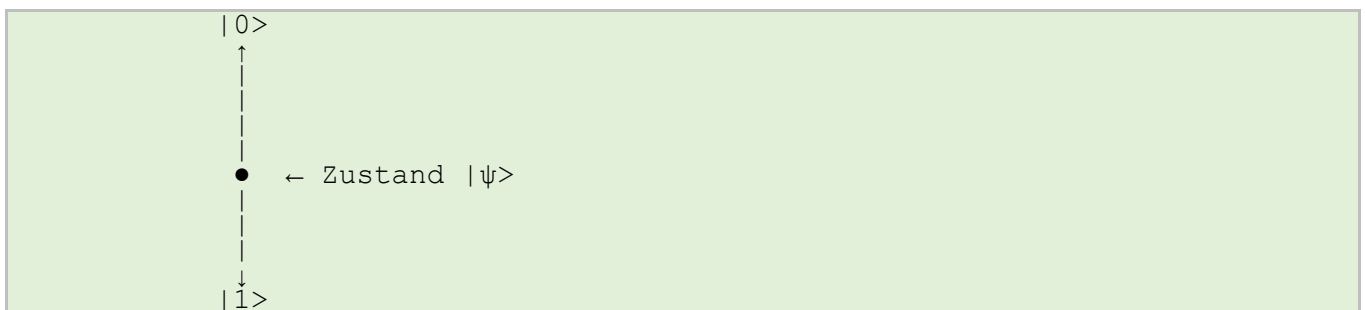
- $\alpha, \beta \in \mathbb{C}$
- $|\alpha|^2 + |\beta|^2 = 1$

Beispiel:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Das ist die **gleichgewichtige Superposition**.

3.4 ASCII-Diagramm: Superposition auf der Bloch-Kugel (vereinfacht)



Die Bloch-Kugel ist ein Modell, das zeigt:

- jeder reine Qubit-Zustand ist ein Punkt auf der Kugel
- Superpositionen sind Punkte zwischen den Polen

Wir bleiben bewusst bei der vereinfachten Darstellung, um das Skript zugänglich zu halten.

3.5 Wie erzeugt man Superposition?

Das wichtigste Werkzeug ist das **Hadamard-Gate**:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Wirkung:

$$\begin{aligned} H|0\rangle &= (|0\rangle + |1\rangle) / \sqrt{2} \\ H|1\rangle &= (|0\rangle - |1\rangle) / \sqrt{2} \end{aligned}$$

Das Hadamard-Gate ist der „Superpositionsgenerator“.

3.6 Interferenz – das Geheimnis der Quantenmechanik

Superposition allein ist noch nicht spannend.

Spannend wird es erst, wenn Superpositionen **interferieren**.

Interferenz bedeutet:

- Amplituden können sich **verstärken** (konstruktiv)
- oder **auslöschen** (destruktiv)

Beispiel:

$$\begin{aligned} &(|0\rangle + |1\rangle) / \sqrt{2} \\ &+ \\ &(|0\rangle - |1\rangle) / \sqrt{2} \\ &= \\ &\sqrt{2} |0\rangle \end{aligned}$$

Die $|1\rangle$ -Anteile löschen sich aus $\rightarrow$ reine Interferenz.

Das ist der Mechanismus hinter:

- Grover
- Shor
- Quantenrandomness

- Bell-Korrelationen
- GHZ-Zuständen

3.7 Beispiel: Superposition im Framework

```
var reg = new QuantumRegister(1);  
Gates.ApplyHadamard(reg, 0);  
  
var result = reg.Measure();  
// liefert 0 oder 1 mit je 50% Wahrscheinlichkeit
```

Für mehrere Qubits:

```
var reg = new QuantumRegister(5);  
for (int q = 0; q < 5; q++)  
    Gates.ApplyHadamard(reg, q);  
  
var bits = reg.MeasureBits();  
// liefert 5 zufällige Bits
```

Das ist bereits ein vollwertiger Quanten-Zufallsgenerator.

3.8 Superposition ist kein „gleichzeitig 0 und 1“

Ein wichtiger Satz:

Superposition bedeutet nicht, dass ein Qubit gleichzeitig 0 und 1 ist. Es bedeutet, dass der Zustand mathematisch beide Möglichkeiten enthält, und *erst die Messung* entscheidet, welche Realität sichtbar wird.

Das ist die korrekte, moderne Formulierung.

Kapitel 4: Tensorprodukte

Tensorprodukte sind das **mathematische Werkzeug**, mit dem man einzelne Qubits zu grösseren Quantenregistern kombiniert.

Das Tensorprodukt ist also eine mathematische Struktur, die beschreibt, wie mehrere Qubits gemeinsam existieren. Ohne Tensorprodukt gäbe es keine Mehr-Qubit-Physik.

Ohne Tensorprodukt gäbe es:

- keine Mehr-Qubit-Register
- keine Bell-Zustände
- keine GHZ-Zustände
- keine Quantenalgorithmen
- keine Verschränkung

Tensorprodukte sind das **Fundament** der Quanteninformatik.

4.1 Intuitive Erklärung

Man stelle sich zwei Qubits vor:

$$\begin{aligned}\text{Qubit A: } |\psi\rangle &= \alpha|0\rangle + \beta|1\rangle \\ \text{Qubit B: } |\varphi\rangle &= \gamma|0\rangle + \delta|1\rangle\end{aligned}$$

Werden die **kombiniert**, entsteht ein **4-dimensionaler Zustand**:

$$|\psi\rangle \otimes |\varphi\rangle = \alpha \gamma |00\rangle + \alpha \delta |01\rangle + \beta \gamma |10\rangle + \beta \delta |11\rangle$$

Das ist das Tensorprodukt.

Es bedeutet:

Das Tensorprodukt bedeutet: Die Amplituden der kombinierten Zustände entstehen durch Multiplikation der Einzelamplituden.

4.2 Warum Tensorprodukt?

Weil ein Quantenregister mit **n Qubits** nicht einfach „n einzelne Qubits“ ist.

Es ist ein **einzigster Zustand** im Raum der Dimension:

$$2^n$$

Beispiele:

- 1 Qubit $\rightarrow$ 2 Amplituden

- 2 Qubits → 4 Amplituden
- 3 Qubits → 8 Amplituden
- 10 Qubits → 1024 Amplituden
- 20 Qubits → 1.048.576 Amplituden

Das ist der Grund, warum Quantencomputer so mächtig sind - und warum Simulationen so teuer werden.

4.2.1 Was das Tensorprodukt physikalisch bedeutet

Das Tensorprodukt ist nicht nur eine mathematische Schreibweise, sondern beschreibt die **physikalische Realität**, wie mehrere Qubits gemeinsam existieren. Es erfüllt drei grundlegende Aufgaben:

1) *Das Tensorprodukt kombiniert Qubits zu einem einzigen Gesamtzustand*

Ein einzelnes Qubit lebt in einem 2-dimensionalen Raum. Zwei Qubits leben **nicht** in zwei getrennten Räumen, sondern gemeinsam in einem **4-dimensionalen Raum**:

$$|\psi\rangle \otimes |\varphi\rangle \in \mathbb{C}^4$$

Das bedeutet:

- Mehrere Qubits bilden **immer einen einzigen Zustand**
- Dieser Zustand hat **2^n Amplituden**
- Alle Qubits sind mathematisch miteinander verbunden

Ohne Tensorprodukt gäbe es **keine Mehr-Qubit-Register**.

2) *Das Tensorprodukt definiert, was Verschränkung überhaupt ist*

Ein Zustand ist **nicht verschränkt**, wenn er sich als Tensorprodukt schreiben lässt:

$$|\psi\rangle \otimes |\varphi\rangle$$

Ein Zustand ist **verschränkt**, wenn das nicht möglich ist, weil

$$|\Phi+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

.... dieser Zustand kann **nicht** geschrieben werden als:

$$|\psi\rangle \otimes |\phi\rangle$$

→ **Das ist die Definition von Verschränkung.**

Das Tensorprodukt ist also nicht nur ein Werkzeug — es ist der **Massstab**, der entscheidet, ob ein Zustand verschränkt ist.

3) Das Tensorprodukt erzeugt die exponentielle Dimension (2^n)

Jedes zusätzliche Qubit verdoppelt die Dimension des Zustandsraums:

- 1 Qubit → 2 Amplituden
- 2 Qubits → 4 Amplituden
- 3 Qubits → 8 Amplituden
- n Qubits → 2^n Amplituden

Das ist der Grund, warum:

- Quantencomputer so mächtig sind
- Simulationen so teuer werden
- Algorithmen wie Shor und Grover funktionieren

Ohne Tensorprodukt gäbe es **keine Quantenparallelität**.

Kurz zusammengefasst

Das Tensorprodukt ist die mathematische Struktur, die beschreibt, wie mehrere Qubits gemeinsam existieren. Es erzeugt Mehr-Qubit-Register, ermöglicht Verschränkung und führt zur exponentiellen Dimension des Zustandsraums.

Oder noch kürzer:

Tensorprodukt = Mehr-Qubit-Physik.

Verschränkung = Nicht-zerlegbares Tensorprodukt.

4.3 Mathematische Definition

Für zwei Vektoren:

$$|\psi\rangle = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, |\phi\rangle = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}$$

ist das Tensorprodukt:

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix} a_0b_0 \\ a_0b_1 \\ a_1b_0 \\ a_1b_1 \end{pmatrix}$$

Für Matrizen funktioniert es genauso - aber das brauchen wir später nur indirekt.

4.4 ASCII-Diagramm: Tensorprodukt

Qubit A: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

Qubit B: $|\varphi\rangle = \gamma|0\rangle + \delta|1\rangle$

Tensorprodukt:

$$|\psi\rangle \otimes |\varphi\rangle =$$

$$\begin{aligned} & \alpha \gamma |00\rangle \\ & + \alpha \delta |01\rangle \\ & + \beta \gamma |10\rangle \\ & + \beta \delta |11\rangle \end{aligned}$$

4.5 Beispiel: Zwei Hadamard-Qubits

Ein einzelnes Hadamard-Qubit:

$$H|0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

Zwei davon:

$$\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \otimes \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)$$

ergibt:

$$\frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Das ist die gleichmässige Superposition über alle 4 Basiszustände.

4.6 Tensorprodukt im Framework

Im Framework ist das Tensorprodukt bereits implizit eingebaut:

- Ein QuantumRegister(n) hat automatisch 2^n Amplituden
- Die Basiszustände sind durch die Bitmuster der Indizes definiert
- Die Gates arbeiten direkt auf diesen Amplituden

Wenn man explizit Tensorprodukte brauchst:

```
Cx[] combined = QuantumMath.Tensor(stateA, stateB);
```

Das ist exakt das mathematische Tensorprodukt.

4.7 Warum Tensorprodukte Verschränkung ermöglichen

Ein Zustand ist **verschränkt**, wenn er **nicht** als Tensorprodukt geschrieben werden kann.

Beispiel:

$$|\Phi+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Kann man **nicht** schreiben als:

$$|\psi\rangle \otimes |\phi\rangle$$

Das ist Verschränkung.

Tensorprodukte sind also nicht nur ein Werkzeug - sie definieren, was Verschränkung überhaupt bedeutet.

4.8 Beispiel: GHZ-Zustand

GHZ₃:

$$(|000\rangle + |111\rangle) / \sqrt{2}$$

Kann man **nicht** als Tensorprodukt schreiben. Das ist maximale Mehr-Qubit-Verschränkung.

4.9 Kriterium für Produktzustand bzw. Verschränkung bei zwei Qubits

Bei einem 2-Qubit-Zustand, kann über die Determinante geprüft werden, ob es sich um ein Produktzustand, oder um eine Verschränkung handelt, wobei hier der 2-Qubit-Zustand ein Spezialfall bildet. Für andere Mehr-Qubit-Zustände gibt es diesen Weg nicht.

Ein allgemeiner 2-Qubit-Zustand kann immer in folgender der Form geschrieben werden.

$$|\Psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$$

Um zu entscheiden, ob dieser Zustand **verschränkt** oder **nicht verschränkt** ist, betrachtet man die **Koeffizientenmatrix**:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Kriterium:

Wenn folgendes gilt, dann ist der Zustand **nicht verschränkt** (Produktzustand).

$$\det(A) = ad - bc = 0,$$

Hingegen hier ist der Zustand **verschränkt**.

$$\det(A) \neq 0,$$

Dieses einfache Determinanten-Kriterium funktioniert für *alle 2-Qubit-Zustände*.

Beispiel: Produktzustand

Ein Produktzustand hat die Form:

$$|\psi\rangle \otimes |\phi\rangle = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$$

Die daraus abgebildete Koeffizientenmatrix ist:

$$A = \begin{pmatrix} \alpha\gamma & \alpha\delta \\ \beta\gamma & \beta\delta \end{pmatrix}$$

Die daraus gebildete Determinante ist:

$$\det(A) = \alpha\gamma \cdot \beta\delta - \alpha\delta \cdot \beta\gamma = 0$$

→ Dieser Zustand ist also immer **nicht verschränkt**.

Kann man aus einem Produktzustand eine Verschränkung machen?

Ja - aber **nicht durch Umformen der Gleichung**.

Verschränkung entsteht nur durch **nicht-lokale unitäre Operationen**, also durch Gates, die beide Qubits gleichzeitig betreffen (z. B. CNOT).

Beispiel:

1. Startzustand (Produkt):

$$|00\rangle$$

2. Hadamard auf Qubit 0:

$$(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

→ immer noch **nicht verschränkt**, Determinante = 0

3. CNOT (0 → 1):

$$\text{CNOT}(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

→ **jetzt verschränkt**

Wichtig: Lokale Operationen $U_A \otimes U_B$ können **nie** Verschränkung erzeugen.

Kann man aus einer Verschränkung wieder ein Tensorprodukt machen?

Wenn der Zustand **wirklich verschränkt** ist → **nein**. Dann existieren keine $|\psi\rangle$ und $|\phi\rangle$, wie das hier:

$$|\Psi\rangle = |\psi\rangle \otimes |\phi\rangle$$

Wenn der Zustand **nicht verschränkt** ist, aber nur „kompliziert aussieht“, kann man ihn durch Faktorisierung wieder in die Form bringen.

$$|\psi\rangle \otimes |\phi\rangle$$

Das Determinanten-Kriterium zeigt bei einem 2-Qubit-Zustand, welcher Fall vorliegt.

Kurz gesagt:

- Produktzustand $\rightarrow \det(A) = 0$
- Verschränkung $\rightarrow \det(A) \neq 0$
- Verschränkung entsteht nur durch **entangling gates**
- Ein verschränkter Zustand kann **nicht** in ein Tensorprodukt zerlegt werden

Kapitel 5: Quantenregister

5.1 Was ist ein Quantenregister? (Intuitive Erklärung)

Ein Quantenregister ist die Sammlung mehrerer Qubits, die gemeinsam einen Zustand bilden.

Ein *klassisches* Register mit 3 Bits kann nur einen der 8 möglichen Zustände annehmen:

000
001
010
011
100
101
110
111

Ein **Quantenregister** mit 3 Qubits kann dagegen **in einer Superposition aller 8 Zustände gleichzeitig** sein:

$$|\Psi\rangle = a_0|000\rangle + a_1|001\rangle + \dots + a_7|111\rangle$$

Das ist der Kern der Quanteninformatik.

5.2 Warum hat ein n-Qubit-Register 2^n Amplituden?

Weil jedes Qubit zwei Basiszustände hat:

$|0\rangle$ oder $|1\rangle$

Für n Qubits:

$$2 \times 2 \times 2 \times \dots \times 2 = 2^n$$

Beispiele:

Qubits	Basiszustände	Amplituden
1	2	2
2	4	4
3	8	8
10	1024	1024
20	1.048.576	1.048.576

Das ist der Grund, warum Quantencomputer so mächtig sind - und warum Simulationen so teuer werden.

5.3 Mathematische Darstellung eines Registers

Ein Register mit n Qubits ist ein Vektor der Länge 2^n :

$$|\psi\rangle = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{2^n - 1} \end{pmatrix}$$

Jeder Index entspricht einem Bitmuster:

Index 0 $\rightarrow$ |000...0>
Index 1 $\rightarrow$ |000...1>
Index 2 $\rightarrow$ |000..10>
...
Index $2^n - 1 \rightarrow$ |111...1>

5.4 ASCII-Diagramm: Register und Basiszustände

Register mit 3 Qubits:

Index	Zustand	Amplitude
0	000	a0
1	001	a1
2	010	a2
3	011	a3
4	100	a4
5	101	a5
6	110	a6
7	111	a7

5.5 Wie speichert man ein Register in der Simulation?

Ein Register ist einfach ein **Array komplexer Zahlen**:

```
Cx[] amplitudes = new Cx[1 << n]; // 2^n
```

Der Startzustand ist:

```
|00...0> = Amplitudes[0] = 1  
alle anderen = 0
```

Das entspricht der physikalischen Realität:

Ein Quantencomputer startet immer in einem definierten Grundzustand.

5.6 Normierung des Registers

Wie bei einem einzelnen Qubit gilt:

$$\sum_i |a_i|^n = 1$$

Im Framework:

```
double sum = 0;  
for (int i = 0; i < amps.Length; i++)  
    sum += amps[i].NormSquared;  
  
double inv = 1.0 / Math.Sqrt(sum);  
  
for (int i = 0; i < amps.Length; i++)  
    amps[i] = amps[i].Scale(inv);
```

5.7 Messung eines Registers

Eine Messung liefert **einen einzigen Basiszustand**, z. B.:

```
10110
```

Die Wahrscheinlichkeit für den Zustand i ist:

$$P(i) = |a_i|^2$$

Im Framework:

```
double r = Random.Shared.NextDouble();
for (int i = 0; i < amps.Length; i++)
{
    if (r < amps[i].NormSquared)
        return i;
    r -= amps[i].NormSquared;
}
```

Das ist exakt die Methode, die auch im libquantum-PDF beschrieben wird (Referenz: Butscher & Weimer, Uni Stuttgart) siehe unten, unter Quellen.

5.8 Warum Register so wichtig sind

Ein Register ist die Grundlage für:

- **Gatter** (Hadamard, CNOT, Toffoli, CCZ, ...)
- Verschränkung
- GHZ-Zustände
- Bell-Zustände
- Quantenalgorithmen
- Quantenrandomness
- Simulationstechniken

Ohne Register gibt es keine Quanteninformatik.

5.9 Beispiel: Ein 3-Qubit-Register im Framework

```
var reg = new QuantumRegister(3);

// Hadamard auf alle Qubits
for (int q = 0; q < 3; q++)
    Gates.ApplyHadamard(reg, q);
```

```
// Messung
var bits = reg.MeasureBits();
Console.WriteLine(string.Join("", bits.Reverse()));
```

Das erzeugt eine **gleichverteilte Superposition über 8 Zustände**.

Kapitel 6: Quantengatter

Quanten-Gatter sind die **Operationen**, mit denen man den Zustand eines Quantenregisters verändert.

Sie sind das Quanten-Äquivalent zu klassischen Logikgattern wie AND, OR, NOT — aber viel mächtiger.

6.1 Was ist ein Quanten-Gate? (Intuitive Erklärung)

Ein Quanten-Gate ist eine **Transformation**, die den Zustand eines Qubits oder eines Registers verändert.

Ein klassisches Gate:

$$\begin{aligned}\text{NOT}(0) &= 1 \\ \text{NOT}(1) &= 0\end{aligned}$$

Ein Quanten-Gate:

$$X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle$$

Das Gate wirkt also **auf die Amplituden**, nicht auf feste Werte.

6.2 Warum müssen Quanten-Gatter unitär sein?

Ein Gate muss:

- **umkehrbar** sein
- **Wahrscheinlichkeiten erhalten**
- **keine Information zerstören**

Das ist nur möglich, wenn das Gate eine **unitäre Matrix** ist:

$$U^\dagger U = I$$

Das bedeutet:

- Spalten sind orthonormal
- das Gate ist invertierbar
- die Norm des Zustands bleibt erhalten

6.3 1-Qubit-Gatter

Ein 1-Qubit-Gate ist eine **2×2-Matrix**:

$$U = \begin{pmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{pmatrix}$$

Wirkung:

$$U(\alpha|0\rangle + \beta|1\rangle) = (\alpha u_{00} + \beta u_{01})|0\rangle + (\alpha u_{10} + \beta u_{11})|1\rangle$$

6.4 Wichtige 1-Qubit-Gatter

Hadamard-Gate (H)

Erzeugt Superposition:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Wirkung:

$$\begin{aligned} H|0\rangle &= (|0\rangle + |1\rangle) / \sqrt{2} \\ H|1\rangle &= (|0\rangle - |1\rangle) / \sqrt{2} \end{aligned}$$

Pauli-X (NOT-Gate)

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Vertauscht $|0\rangle$ und $|1\rangle$.

Pauli-Z (Phasenflip)

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Lässt $|0\rangle$ unverändert, gibt $|1\rangle$ ein Minuszeichen.

6.5 2-Qubit-Gatter

Ein 2-Qubit-Gate ist eine **4×4-Matrix**. Das wichtigste davon ist:

CNOT (Controlled-NOT)

- Wenn Control = 1 → flippe Target
- Wenn Control = 0 → tue nichts

Matrix:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Wirkung:

$$\begin{aligned} |00\rangle &\rightarrow |00\rangle \\ |01\rangle &\rightarrow |01\rangle \\ |10\rangle &\rightarrow |11\rangle \\ |11\rangle &\rightarrow |10\rangle \end{aligned}$$

CNOT ist das Gate, das **Verschränkung** erzeugt.

6.6 3-Qubit-Gatter

Toffoli (CCNOT)

- Zwei Controls
- Ein Target
- Wenn beide Controls = 1 → flippe Target

Das ist das reversible Pendant zum klassischen AND-Gate.

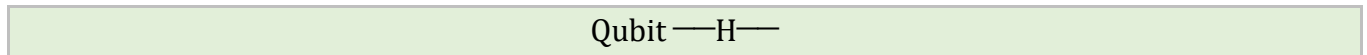
CCZ (Controlled-Controlled-Z)

- Zwei Controls
- Ein Target
- Wenn alle drei Qubits = 1 $\rightarrow$ Phase -1

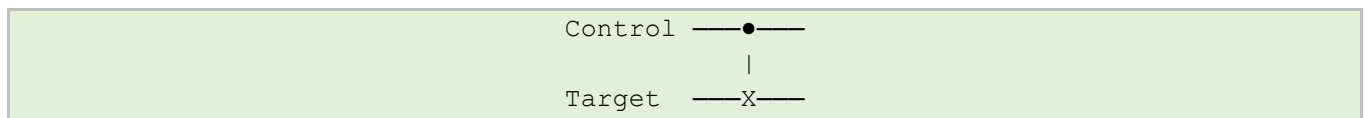
Wird in vielen Algorithmen verwendet.

6.7 ASCII-Diagramme für GitHub

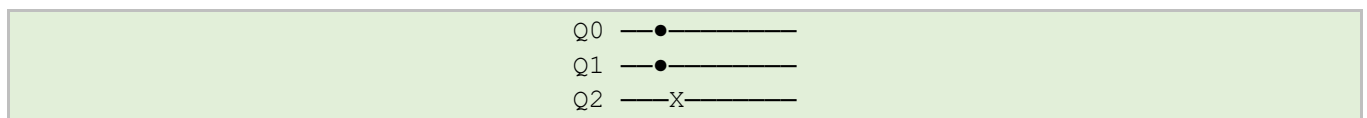
Hadamard



CNOT



Toffoli



6.8 Wie wirken Gatter auf Register?

Ein Gate wirkt nicht auf einzelne Qubits, sondern auf die Amplituden des gesamten Registers.

Beispiel:

Hadamard auf Qubit 0 eines 3-Qubit-Registers:

- wirkt auf Paare von Amplituden
- deren Index sich nur im LSB unterscheidet

Beispiel:

CNOT(control=1, target=3) eines 5-Qubit-Registers:

- wirkt auf alle Amplituden, deren Bit 1 = 1
- und flippt Bit 3

Das ist exakt, was das Framework implementieren muss.

6.9 Gatter im Framework

Hadamard

```
Gates.ApplyHadamard(reg, targetQubit);
```

Pauli-X

```
Gates.ApplyPauliX(reg, targetQubit);
```

Pauli-Z

```
Gates.ApplyPauliZ(reg, targetQubit);
```

CNOT

```
Gates.ApplyCnot(reg, control, target);
```

Diese Implementationen sind **sparse**, also extrem effizient.

Kapitel 7: Messungen (Measure)

Die Messung ist der entscheidende Schritt, der Quantenmechanik von klassischer Physik unterscheidet.

Sie ist der Moment, in dem ein Quantenregister „eine Entscheidung trifft“.

7.1 Was bedeutet Messung? (Intuitive Erklärung)

Ein Qubit kann in einem Zustand wie:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

sein — also in einer Superposition.

Wenn man es misst, passiert Folgendes:

- Man bekommst 0 mit Wahrscheinlichkeit $|\alpha|^2$
- Man bekommst 1 mit Wahrscheinlichkeit $|\beta|^2$
- Der Zustand kollabiert in den gemessenen Zustand

Das bedeutet:

Die Messung zerstört die Superposition und erzeugt ein klassisches Ergebnis.

7.2 Warum kollabiert der Zustand?

Das ist keine „magische“ Eigenschaft, sondern eine **mathematische Regel** der Quantenmechanik:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Nach einer Messung mit Ergebnis 0 wird der Zustand:

$$|0\rangle$$

Nach einer Messung mit Ergebnis 1 wird der Zustand:

$$|1\rangle$$

Die ursprünglichen Amplituden sind weg.

7.3 Messung eines n-Qubit-Registers

Ein Register mit n Qubits hat 2^n Amplituden:

$$|\Psi\rangle = \sum_{i=0}^{2^n-1} a_i |i\rangle$$

Die Wahrscheinlichkeit, den Zustand i zu messen, ist:

$$P(i) = |a_i|^2$$

Beispiel für 3 Qubits:

$$\begin{aligned} &a_0 |000\rangle \\ &a_1 |001\rangle \\ &a_2 |010\rangle \\ &\dots \\ &a_7 |111\rangle \end{aligned}$$

Wenn man misst, bekommst man **genau einer** dieser 8 Zustände.

7.4 ASCII-Diagramm: Messung

Vor der Messung:

$$|\psi\rangle = a_0|000\rangle + a_1|001\rangle + \dots + a_7|111\rangle$$

Messung:

Ergebnis: z.B. 101

Nach der Messung:

$$|\psi\rangle = |101\rangle$$

7.5 Warum ist Messung irreversibel?

Weil durch die Messung Information verloren gehen.

Vor der Messung:

- man kennt alle Amplituden
- man kennt die gesamte Superposition
- man kennt die Phasen

Nach der Messung:

- man kennt nur noch ein einziges Bitmuster
- alle anderen Informationen sind weg

Das ist der Grund, warum Quantenalgorithmen so clever sein müssen:

Sie müssen die richtige Information in die Messung hineininterferieren.

7.6 Beispiel: Messung eines Bell-Zustands

Bell-State:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Messung liefert:

- 00 mit 50 %
- 11 mit 50 %
- niemals 01 oder 10

Das ist Verschränkung.

7.7 Beispiel: Messung eines GHZ-Zustands

$$(|000\rangle + |111\rangle)/2$$

Messung liefert:

- 000
- oder 111
- niemals etwas anderes

Das ist Mehr-Qubit-Verschränkung.

7.8 Messung im Framework

Ganzes Register messen

```
int index = reg.Measure();
```

Bits extrahieren

```
byte[] bits = reg.MeasureBits();
```

Interpretation

- *index* ist ein Integer zwischen 0 und $2^n - 1$
- *bits* ist ein Array der gemessenen Qubit-Werte

Beispiel:

```
index = 13  
bits = [1, 0, 1, 1] // LSB-first
```

7.9 Warum Messung so wichtig ist

Messung ist der **Ausgang** eines Quantenalgorithmus.

Ein Algorithmus ist erfolgreich, wenn:

- die richtige Antwort eine **hohe Wahrscheinlichkeit** hat
- falsche Antworten **ausgelöscht** wurden (destruktive Interferenz)

Das ist das Geheimnis von:

- Grover
- Shor
- Quantenrandomness
- GHZ- und Bell-Experimenten

Kapitel 8: GHZ- und Bell-Zustände

GHZ- und Bell-Zustände sind die klassisch nicht erklärbaren Zustände der Quantenmechanik.

Sie sind das, was Quantencomputer wirklich besonders macht: Verschränkung.

Dieses Kapitel ist ein Highlight, weil es zeigt:

- wie Verschränkung entsteht
- wie man sie erzeugt
- wie man sie misst
- wie man sie simuliert
- wie sie im Framework funktioniert

8.1 Was ist Verschränkung? (Intuitive Erklärung)

Zwei Qubits sind verschränkt, wenn sie nicht mehr unabhängig voneinander beschrieben werden können.

Beispiel:

$$\begin{aligned} &\text{Nicht verschränkt:} \\ |\psi\rangle &= (\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle) \end{aligned}$$

Das ist einfach ein Produkt zweier Einzelzustände.

Verschränkt:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Dieser Zustand kann **nicht** geschrieben werden als:

$$|\psi\rangle \otimes |\phi\rangle$$

Das ist Verschränkung.

8.2 Warum ist Verschränkung so besonders?

Weil sie Eigenschaften hat, die klassisch unmöglich sind:

- Messung eines Qubits beeinflusst das andere
- Korrelationen sind stärker als klassisch erlaubt
- keine klassische Erklärung möglich
- Grundlage für:
 - Quantenkommunikation
 - Teleportation
 - Superdense Coding
 - Quantenrandomness
 - Quantenalgorithmen

Verschränkung ist das „Betriebssystem“ der Quanteninformatik.

8.3 Die vier Bell-Zustände

Bell-Zustände sind die **einfachste Form von Verschränkung** – zwei Qubits, maximal verschränkt.

Es gibt genau vier:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

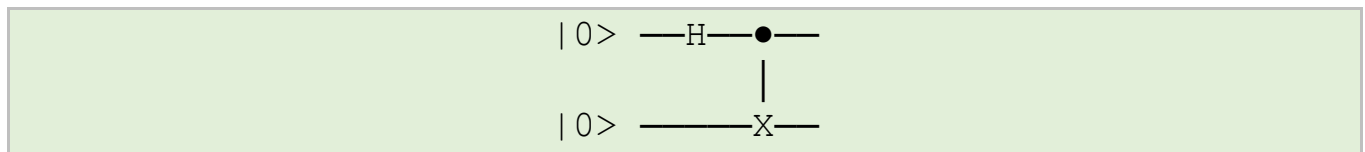
$$|\Phi-\rangle = \frac{1}{\sqrt{2}} (|00\rangle - |11\rangle)$$

$$|\Psi+\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |10\rangle)$$

$$|\Psi-\rangle = \frac{1}{\sqrt{2}} (|01\rangle - |10\rangle)$$

8.4 ASCII-Diagramm: Erzeugung eines Bell-Zustands

Der Standardweg:



Schritte:

1. Hadamard auf Qubit 0
2. CNOT(control=0, target=1)

Ergebnis:

$$|\Phi+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

8.5 Messung eines Bell-Zustands

Für $|\Phi+\rangle$:

- $00 \rightarrow 50\%$
- $11 \rightarrow 50\%$
- $01 \rightarrow 0\%$
- $10 \rightarrow 0\%$

Das ist die Signatur der Verschränkung.

8.6 Bell-Zustände im Framework

```
var reg = new QuantumRegister(2);  
  
Gates.ApplyHadamard(reg, 0);  
Gates.ApplyCnot(reg, 0, 1);  
  
var bits = reg.MeasureBits();  
Console.WriteLine(string.Join("", bits.Reverse()));
```

Das erzeugt $|\Phi+\rangle$.

8.7 GHZ-Zustände (Mehr-Qubit-Verschrankung)

GHZ-Zustände sind die Verallgemeinerung der Bell-Zustände auf mehr als zwei Qubits.

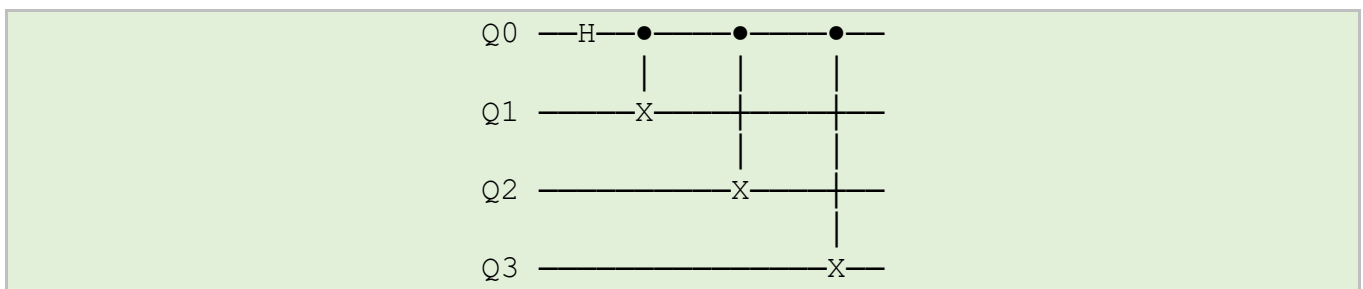
Für n Qubits:

$$|\text{GHZ}_n\rangle = \frac{|00 \dots 0\rangle + |11 \dots 1\rangle}{\sqrt{2}}$$

Beispiele:

- $\text{GHZ}_2 = \text{Bell-State}$
- $\text{GHZ}_3 = (|000\rangle + |111\rangle)/\sqrt{2}$
- $\text{GHZ}_4 = (|0000\rangle + |1111\rangle)/\sqrt{2}$

8.8 ASCII-Diagramm: GHZ-Erzeugung



Schritte:

1. Hadamard auf Qubit 0
2. CNOT von Qubit 0 auf alle anderen

8.9 GHZ-Zustände im Framework

```
var reg = new QuantumRegister(n);

// H auf Qubit 0
Gates.ApplyHadamard(reg, 0);

// CNOT von 0 auf alle anderen
for (int q = 1; q < n; q++)
    Gates.ApplyCnot(reg, 0, q);
```

8.10 Messung eines GHZ-Zustands

Für GHZ_3 :

- $000 \rightarrow 50\%$
- $111 \rightarrow 50\%$
- alles andere $\rightarrow 0\%$

Für GHZ_4 :

- $0000 \rightarrow 50\%$
- $1111 \rightarrow 50\%$

GHZ-Zustände sind **maximal verschränkt**.

8.11 Warum GHZ und Bell so wichtig sind

Sie sind die Grundlage für:

- Quantenkommunikation
- Teleportation
- Superdense Coding
- Quantenrandomness
- Tests der Quantenmechanik (Bell-Ungleichungen)
- Quantenalgorithmen
- Quantenfehlerkorrektur

Und sie sind die besten Testzustände für das Framework.

Kapitel 9: Quanten-Zufallszahlen (QRNG)

Echte Quantenmechanik liefert echten Zufall.

Nicht „pseudo-zufällig“, nicht „algorithmisch generiert“, sondern **fundamental unvorhersagbar**.

Ein Quanten-Zufallszahlengenerator (QRNG) nutzt genau diese Eigenschaft.

9.1 Warum Quantenmechanik echten Zufall liefert

Ein Qubit in Superposition:

$$|\psi\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

liefert bei Messung:

- 0 mit 50 %
- 1 mit 50 %

Und das ist **nicht**:

- chaotisch
- deterministisch
- algorithmisch
- pseudo-zufällig

Sondern:

Die Quantenmechanik sagt: Das Ergebnis ist fundamental unbestimmt.
Es gibt keine verborgenen Variablen, die das Ergebnis festlegen.

Das ist durch Experimente (Bell-Tests) bestätigt.

9.2 Wie erzeugt man Quanten-Zufall?

Der einfachste QRNG besteht aus:

- einem Qubit im Zustand $|0\rangle$
- einem Hadamard-Gate
- einer Messung

ASCII-Diagramm:

$|0\rangle \xrightarrow{\text{H}} \bullet \rightarrow 0 \text{ oder } 1 \text{ (je 50\%)}$

9.3 QRNG mit mehreren Qubits

Wenn man n Qubits hat und auf jedes ein Hadamard anwendest:

$$H \otimes n |00 \dots 0\rangle = \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} |i\rangle$$

Das ist die gleichverteilte Superposition über alle 2^n Zustände.

Messung liefert:

- eine Zahl zwischen 0 und $2^n - 1$
- mit gleicher Wahrscheinlichkeit

Das ist ein n-Bit-Zufallswert.

9.4 QPRNG im Framework (einfachste Version)

```
public ulong NextRaw()
{
    var reg = new QuantumRegister(_qubits);

    for (int q = 0; q < _qubits; q++)
        Gates.ApplyHadamard(reg, q);

    return (ulong)reg.Measure();
}
```

Das ist bereits ein vollwertiger QPRNG.

9.5 QRNG mit GHZ-Zuständen

GHZ-Zustände liefern **korrelierte Zufallsbits**:

$$(|00\dots 0\rangle + |11\dots 1\rangle) / 2$$

Messung:

- 000...0
- oder 111...1

→ perfekt korrelierte Bits
→ n-Bit-Zufallswert, aber nur 2 mögliche Werte
→ ideal für Tests von Verschränkung
→ nicht ideal für „gleichverteilten“ Zufall

9.6 QRNG mit Bell-Zuständen

Bell-Zustände liefern:

- 00 oder 11
- 01 oder 10

→ je nach Bell-State andere Korrelationen
→ nützlich für Quantenkommunikation
→ weniger für gleichverteilten Zufall

9.7 Warum QRNG so wichtig ist

Quanten-Zufall wird heute verwendet in:

- Kryptographie
- Schlüsselgenerierung
- Simulationen
- Monte-Carlo-Methoden
- statistischen Tests
- wissenschaftlichen Experimenten

Und das Framework kann das **alles simulieren**.

9.8 Beispiel: 5-Qubit-QPRNG

```
var qrng = new QuantumQrng(5);  
  
for (int i = 0; i < 10; i++)  
{  
    ulong value = qrng.NextRaw();  
    Console.WriteLine(value);  
}
```

Ausgabe:

0..31 (gleichverteilte Zufallszahlen)

9.9 QPRNG als Baustein für das GitHub-Projekt

Ich werde im GitHub-Projekt:

- ein Kapitel „Quanten-Zufall“ einbauen
- Beispiele zeigen
- Diagramme einfügen
- Code-Snippets einbauen
- Tests durchführen
- statistische Verteilungen zeigen

Das macht das Projekt **praktisch**, **spannend** und **wissenschaftlich wertvoll**.

Kapitel 10: Simulationstechniken

Ein echter Quantencomputer arbeitet mit physikalischen Qubits.

Eine Simulation dagegen arbeitet mit Zustandsvektoren, Amplituden und unitären Operationen.

Dieses Kapitel erklärt:

- wie man Quantenregister simuliert
- wie man Gatter effizient anwendet
- warum Simulation teuer wird
- welche Optimierungen möglich sind
- wie das Framework das alles umsetzt

10.1 Warum ist Quanten-Simulation schwierig?

Ein Register mit n Qubits hat **2^n Amplituden**.

Beispiele:

Qubits	Amplituden	Speicherbedarf (double complex)
10	1.024	~16 KB
20	1.048.576	~16 MB
25	33.554.432	~512 MB
30	1.073B	~16 GB

Das bedeutet:

Jeder zusätzliche Qubit verdoppelt den Speicherbedarf.

Deshalb sind klassische Simulationen **exponentiell teuer**.

10.2 Der Zustandsvektor

Die einfachste und am weitesten verbreitete Simulationstechnik ist der State-Vector-Simulator.

Ein Register mit n Qubits wird als Vektor der Länge 2^n gespeichert:

```
Cx[] amplitudes = new Cx[1 << n];
```

Das ist genau, was das Framework macht.

10.3 Anwendung von Gattern (naiv)

Ein 1-Qubit-Gate ist eine 2×2 -Matrix.

Ein 2-Qubit-Gate ist eine 4×4 -Matrix.

Naiv könnte man sagen:

```
Multipliziere die grosse  $2^n \times 2^n$ -Matrix mit dem Zustandsvektor.
```

Aber:

- eine 20-Qubit-Matrix hätte Grösse **$1.048.576 \times 1.048.576$**
- das wären **über eine Billion Einträge**
- völlig unpraktisch

Deshalb macht man **es nicht so**.

10.4 Sparse-Simulation (effizient)

Statt grosse Matrizen zu bauen, nutzt man die Struktur der Gatter:

1-Qubit-Gatter

Wirkt auf Paare von Amplituden:

```
i0 = ...0...
```

```
i1 = ...1...
```

- nur 2 Amplituden werden gemischt
- alle anderen bleiben unverändert

2-Qubit-Gatter (z. B. CNOT)

Wirkt auf Amplituden, deren Bits bestimmte Muster haben:

wenn control = 1 → flippe target

- reine Permutation
- extrem effizient

Das ist exakt, was das Framework implementiert.

10.5 Permutationsgatter

Viele wichtige Gatter sind Permutation:

- X (NOT)
- CNOT
- Toffoli
- SWAP

Permutation bedeutet:

Nur die Positionen der Amplituden werden vertauscht, nicht ihre Werte.

Das ist extrem schnell.

10.6 Phasengatter

Andere Gatter ändern nur die Phase bestimmter Amplituden:

- Z
- CZ
- CCZ
- T-Gate
- S-Gate

Beispiel:

wenn Bit $k = 1 \rightarrow$ multipliziere Amplitude mit -1

Auch das ist extrem effizient.

10.7 Parallelisierung

Da viele Operationen unabhängig sind, kann man:

- *Parallel.For*
- SIMD
- GPU-Beschleunigung

nutzen.

Das Framework ist bereits so strukturiert, dass man später leicht parallelisieren kann.

10.8 Speicheroptimierung

Ein paar Tricks:

- `Span<T>` statt Arrays
- `ref struct` für komplexe Zahlen
- In-Place-Operationen
- Vermeidung von temporären Arrays
- Lazy-Normalisierung

Aber:

Für das Projekt ist das **nicht nötig** – der Code ist bereits effizient genug.

10.9 Grenzen der Simulation

Ein klassischer Computer kann realistisch:

- bis ~25 Qubits problemlos
- bis ~30 Qubits mit viel RAM
- darüber hinaus wird es schwierig

Ein echter Quantencomputer kann:

- 50+ Qubits
- 100+ Qubits
- 1000+ Qubits (Zukunft)

Das zeigt:

Simulation ist ein Werkzeug – aber kein Ersatz für echte Quantenhardware.

10.10 Simulation im Framework

State-Vector-Simulation

→ *QuantumRegister* speichert 2^n Amplituden

Sparse-Gates

→ *ApplyHadamard*, *ApplyPauliX*, *ApplyCnot* arbeiten direkt auf relevanten Amplituden

Permutation & Phase

→ X, CNOT, Toffoli = Permutation

→ Z, CZ, CCZ = Phase

Effiziente Messung

→ Wahrscheinlichkeiten aus Amplituden

→ Zufallszahl → Index

GHZ, Bell, QRNG

→ alles effizient simulierbar

Schlusswort

Mit diesem Skript konnte man die wichtigsten Bausteine der Quanteninformatik kennengelernt werden: Qubits, Amplituden, Superposition, Tensorprodukte, Quantenregister, Gatter, Messung, Verschränkung, GHZ- und Bell-Zustände, Quanten-Zufall und grundlegende Simulationstechniken.

Diese Grundlagen bilden das Fundament für nahezu alle modernen Quantenalgorithmen – von Grover über Shor bis hin zu Quantenkommunikationsprotokollen. Auch wenn echte Quantencomputer heute noch in den Kinderschuhen stecken, ist das Verständnis dieser Konzepte bereits jetzt von grosser Bedeutung. Die Zukunft der Informatik wird zweifellos durch Quantenmechanik mitgeprägt werden.

Die hier vorgestellten Inhalte sollen nicht nur theoretisches Wissen vermitteln, sondern auch zur eigenen praktischen Umsetzung motivieren. Die begleitenden C#-Implementationen zeigen, dass sich viele Konzepte der Quantenmechanik elegant und effizient simulieren lassen – und dass der Einstieg in die Quanteninformatik weniger kompliziert ist, als es auf den ersten Blick scheint.

Ich wünsche viel Freude beim weiteren Erkunden, Experimentieren und Vertiefen.

Die Quantenwelt ist gross – und dies war erst der Anfang.

Autoren- und Quellenhinweis

Dieses Skript entstand im Rahmen eines persönlichen Studien- und Entwicklungsprojekts zum Thema Quantencomputing. Ziel war es, die grundlegenden Konzepte — von Qubits über Superposition und Quantenregister bis hin zu Gattern, Messung und einfachen Simulationstechniken — kompakt, verständlich und praxisnah aufzubereiten.

Der Fokus liegt auf der Konstruktion eines kleinen, konzeptionellen „Mini-Quantencomputers“, der speziell für die Erzeugung kryptographisch sicherer Zufallszahlen gedacht ist. Die Inhalte basieren auf eigener Recherche, praktischen Experimenten sowie ausgewählten Quellen aus der Quanteninformatik.

Dieses Dokument erhebt keinen Anspruch auf Vollständigkeit, soll jedoch als Einstieg, Begleitmaterial und Referenz dienen.

Besonders hilfreich waren folgende Referenzen:

- ***Simulation eines Quantencomputers***
Björn Butscher, Hendrik Weimer
Universität Stuttgart, März 2003
- ***Skript zur Vorlesung „Quanten Computing“***
Georg Hoever
Fachbereich Elektrotechnik und Informationstechnik
FH Aachen
- ***Quanteninformatik - schneppat.de***
Grundlagen Quanteninformatik: Qubits, Superposition,
Verschränkung und Überlagerung, Quantengatter,
Quantenalgorithmen, Bell'sches Theorem,

Autor: Michele Natale

Github: github.com/michelenatale

Projekt: Mini-QuantenComputer

Schweiz, 2026